



UNIVERSIDAD AUTÓNOMA DE YUCATÁN

FACULTAD DE MATEMÁTICAS

Diseño de Software

VitalLogix

Integrantes:

Manuel Ávila Lombardi

José Rodolfo Hernández Escamilla

Linda Natalia Loeza Suaste

Canche Uicab Perla Noemi

Luis Gilberto Dzib Pech

Profesor:

José Luis López Martínez

Fecha de Entrega:

18 de mayo de 2026

Contenido

Introducción	3
Objetivo	3
Requerimientos Funcionales.....	4
Diagrama de casos de uso	5
Diagrama de clases.....	6
Diagrama de secuencia (Venta).....	7
Diagrama de estados (Producto)	8
Diagrama de colaboración	9
Diagrama de actividad (Venta).....	10
Vistas del Producto	11
Interfaz del Administrador	11
Interfaz de inicio de sesión	11
Interfaz de compra por un invitado	12
Evidencias de implementación.....	13
Patrón Observer.....	13
Patrón Strategy	14
Patrón Singleton	15
Ejemplo del funcionamiento del sistema.....	15
Enlace al repositorio en GitHub	15
Conclusiones.....	16

Introducción

VitalLogix es un sistema de gestión para farmacias desarrollado como proyecto final de la asignatura Diseño de Software. La plataforma permite administrar el inventario de productos, registrar ventas, entre otros, todo desde una interfaz web accesible para distintos tipos de usuarios.

Su desarrollo integra los conceptos vistos durante el curso: se aplicaron principios SOLID, patrones de diseño y buenas prácticas de arquitectura de software, estas características deberían ver reflejadas tanto en el diagrama de clases como en el código fuente.

El objetivo de este documento es describir el análisis, modelado y diseño del sistema, incluyendo los diagramas UML correspondientes, la evidencia de los patrones implementados y las vistas del producto final.

Objetivo

El proyecto consiste en un Sistema de Gestión de Farmacia en Java y tiene como objetivo crear una aplicación de software robusta y eficiente que permita a una farmacia gestionar de manera efectiva su inventario, ventas compras y registros de productos. Este sistema brindará una solución para operaciones diarias de una farmacia, mejorando la eficiencia, la precisión y la velocidad de respuesta en la atención al cliente y la toma de decisiones comerciales.

Requerimientos Funcionales

1. Gestión de Inventarios

- Nuestro sistema debe permitir la incorporación de nuevos productos al inventario.
- Debemos permitir la actualización de existencias de productos.
- Debemos registrar la fecha de vencimiento de los productos.
- Debemos permitir la eliminación de productos obsoletos o vencidos.

2. Registro de Ventas

- Nuestro sistema debe permitir la venta de productos al cliente.
- Debemos calcular automáticamente el precio total de la compra.
- Debemos registrar la información del cliente (nombre, dirección, número de contacto) para ventas con receta.
- Debemos generar un recibo para cada venta.

3. Búsqueda y Consulta de Productos

- Debemos permitir la búsqueda rápida de productos por nombre, código o categoría.
- Debemos proporcionar información detallada de cada producto, incluyendo precio, existencias y fecha de vencimiento.

4. Gestión de Clientes

- Debemos permitir la creación y mantenimiento de registros de clientes.
- Debemos proporcionar información sobre las compras anteriores de los clientes.
- Debemos permitir la asignación de descuentos o programas de fidelización a clientes habituales.
- Nuestros clientes deben contar con un número de clienteamigo que les permita acceder a nuestro programa de descuentos.

5. Generación de Reportes

- Debemos ser capaces de generar informes de ventas diarias, semanales, mensuales y anuales.
- Debemos proporcionar informes de inventario actualizados.

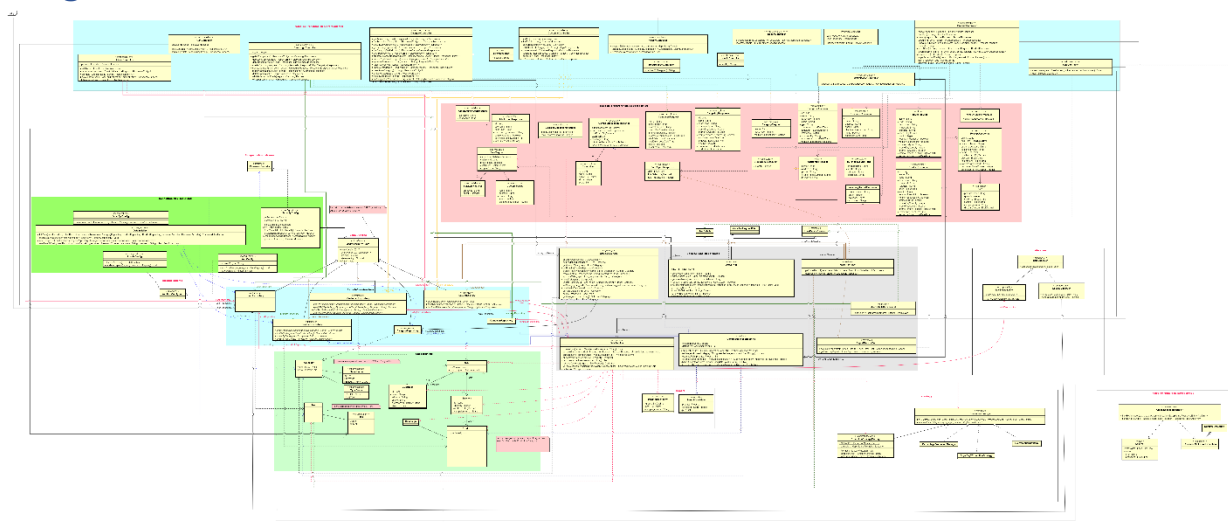
6. Gestión de Categorías

- Nuestro sistema incluye un módulo de categorías con tipos predefinidos y personalizados.
- Permitimos que las categorías personalizadas se envíen para aprobación o rechazo.
- Hacemos que las categorías activas estén disponibles en formularios de producto y filtros de inventario.



<https://github.com/LuisGilDzib/VitalLogix/blob/main/docs/diagrams/UseCase%20VitalLogix.asta>

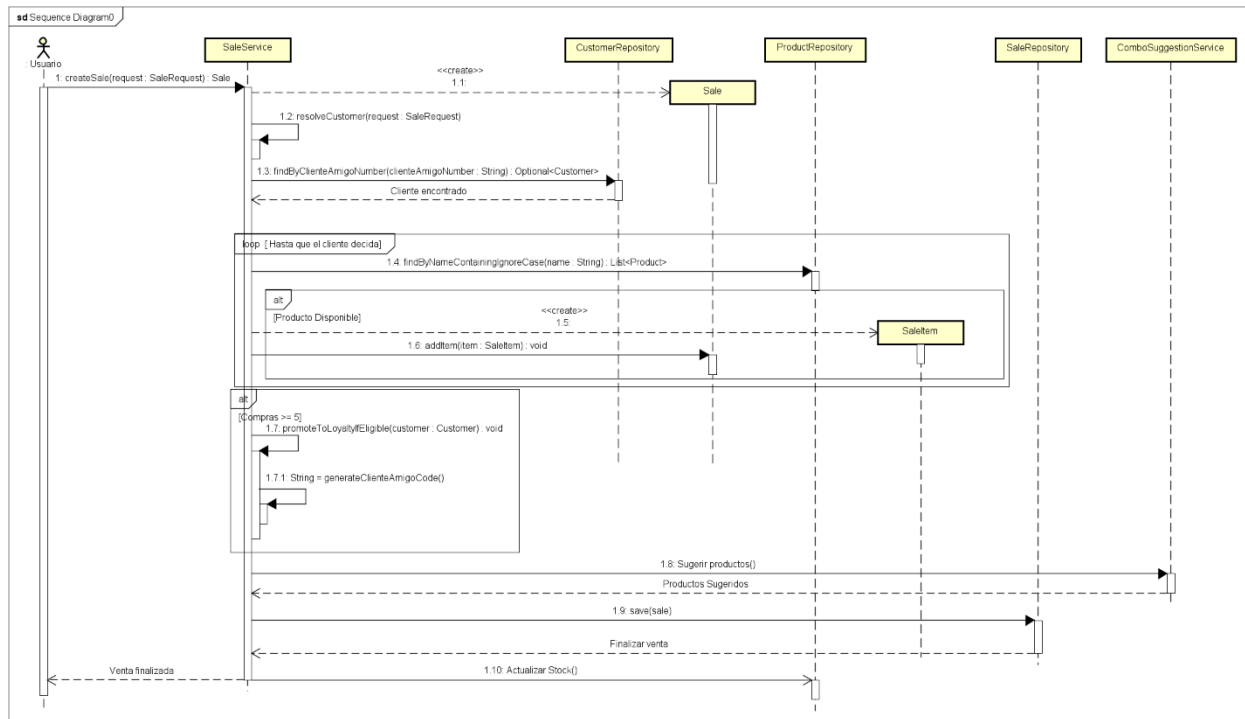
Diagrama de clases



Enlace al diagrama en GitHub:

<https://github.com/LuisGilDzib/VitalLogix/blob/main/docs/diagrams/ClassDiagram%20VitalLogix.asta>

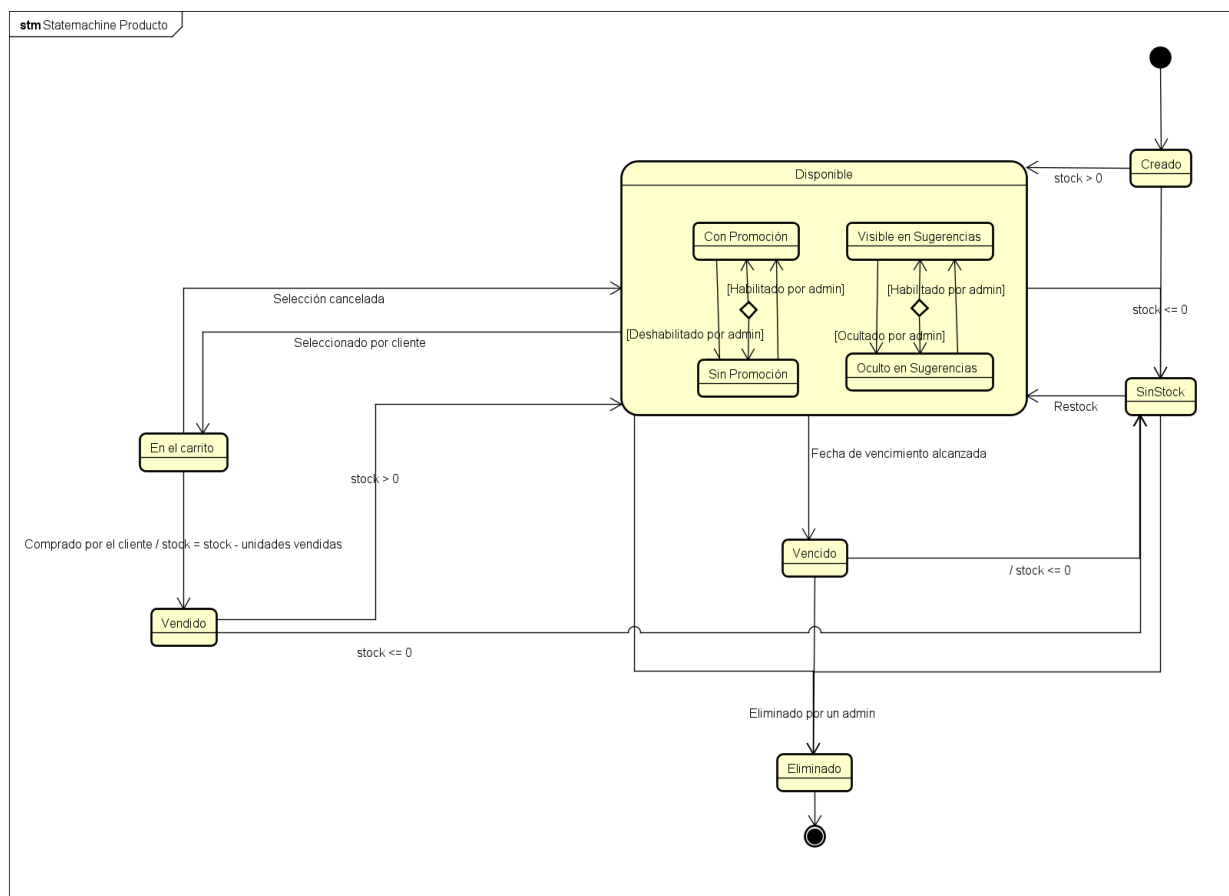
Diagrama de secuencia (Venta)



Enlace al diagrama en GitHub:

<https://github.com/LuisGilDzib/VitalLogix/blob/main/docs/diagrams/SequenceDiagram%20Vitallogix.asta>

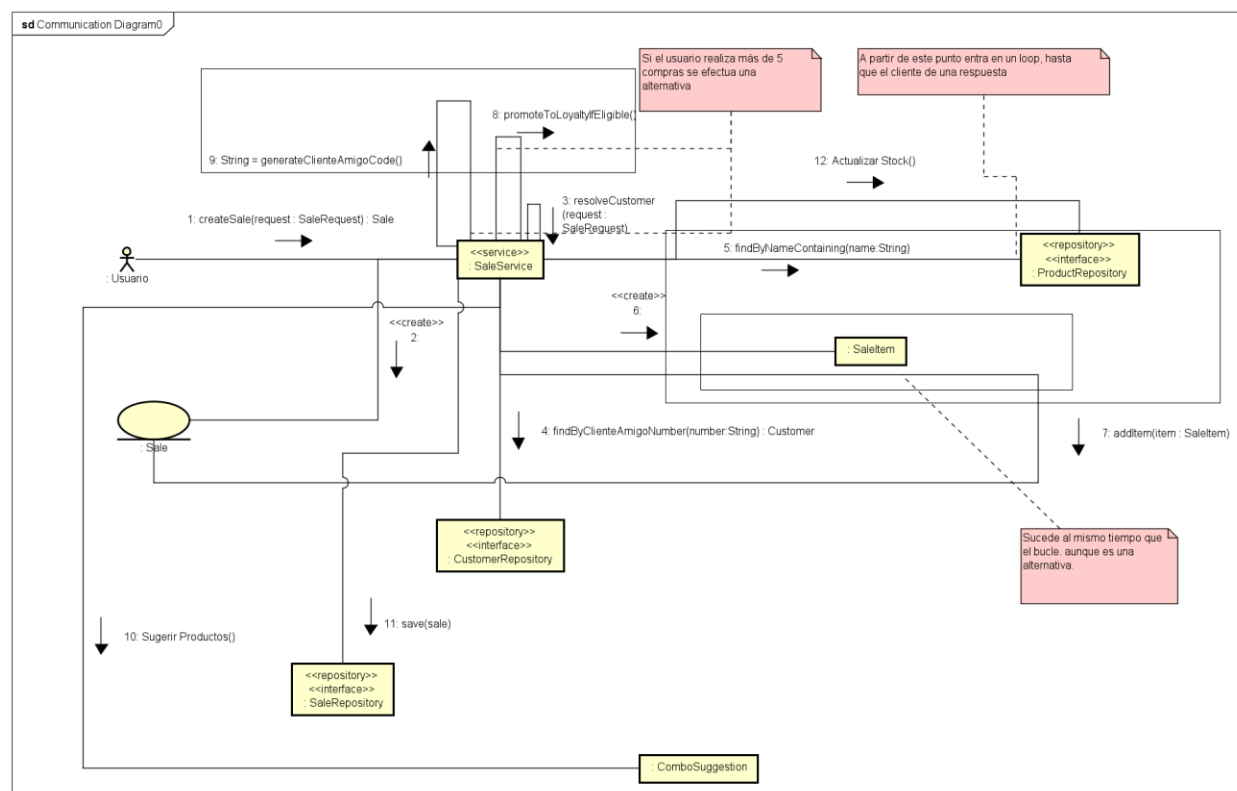
Diagrama de estados (Producto)



Enlace al diagrama en GitHub:

<https://github.com/LuisGilDzib/VitalLogix/blob/main/docs/diagrams/StateMachine%20Producto.asta>

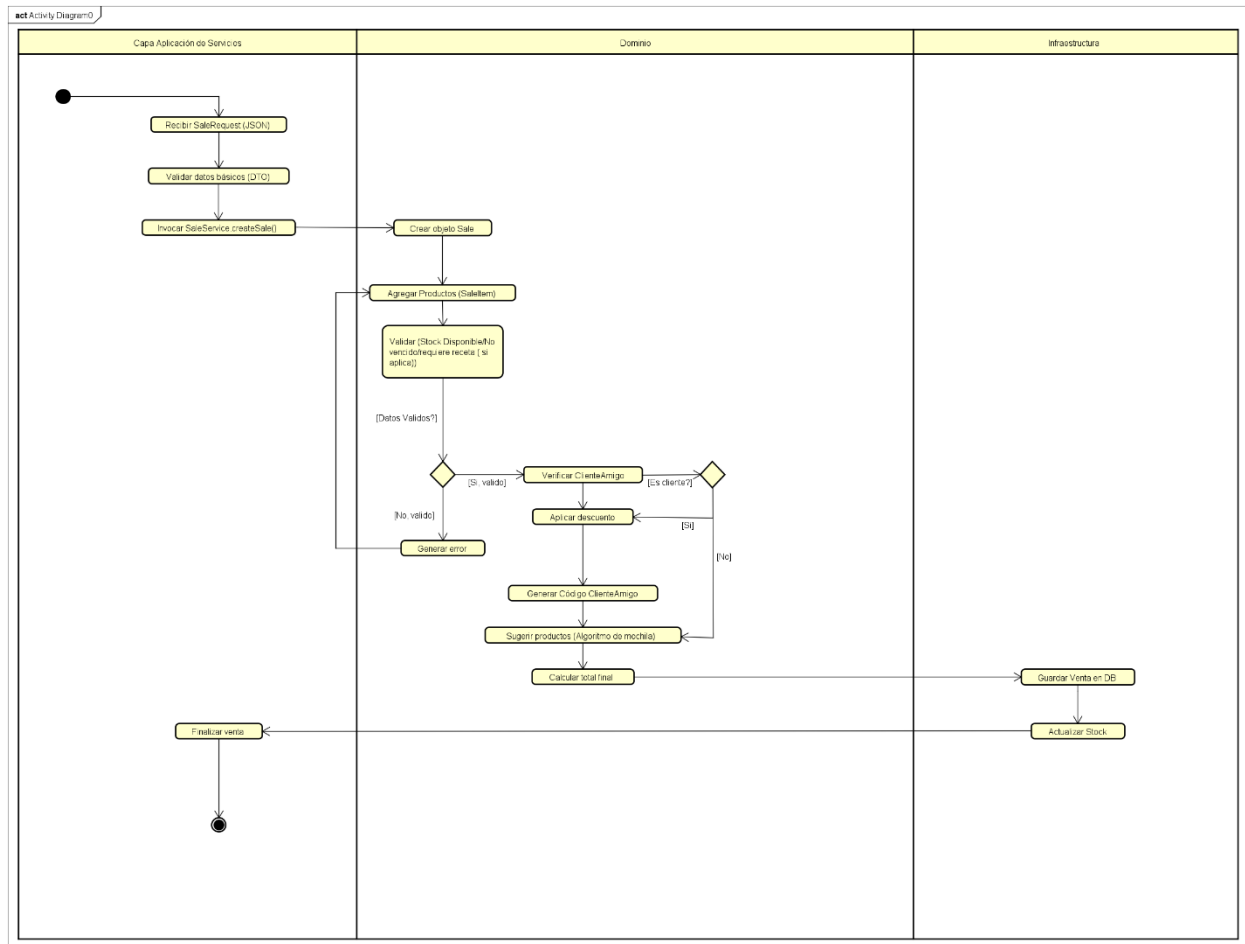
Diagrama de colaboración



Enlace al diagrama en GitHub:

<https://github.com/LuisGilDzib/VitalLogix/blob/main/docs/diagrams/ColaborationDiagram.asta>

Diagrama de actividad (Venta)



Enlace al diagrama en GitHub:

https://github.com/LuisGilDzib/VitalLogix/blob/main/docs/diagrams/Activity%20VitalLogix_asta.asta

Vistas del Producto

Interfaz del Administrador

The screenshot displays the Vitallogix Admin Interface. At the top, there's a header with the logo, user info (admin1, SALIR), and navigation links (VENTAS, HISTORIAL, CLIENTES, CATEGORÍAS). Below the header, there's a section for reports (REPORTES) with filters for DIARIO, SEMANAL, MENSUAL, and ANUAL, and date ranges (09/04/2026 to 09/04/2026). The main area shows a table of products with columns: PRODUCTO, CATEGORIA, STOCK, PRECIO, and VENCIMIENTO. The table lists several products like Electrolit, Leche Nido, Ibuprofeno, paracetamol, Producto Editado QA, Leche NanPro, Suerox, and Insulina. To the right, there's a sidebar with a 'RESUMEN DE VENTA' section, including 'Ver recomendaciones personalizadas' and 'Ver sugerencias' buttons. Below that, there are checkboxes for 'VENTA CON RECETA' and 'APLICAR CLIENTEAMIGO'. At the bottom of the sidebar, it says 'Carrito vacío'.

PRODUCTO	CATEGORIA	STOCK	PRECIO	VENCIMIENTO
Electrolit	SUPLEMENTOS	8	\$15.00	4/9/2029
Leche Nido	BEBIDAS	13	\$120.00	6/9/2031
Ibuprofeno	ANTIBIÓTICOS	4	\$120.00	6/5/2030
paracetamol	ANTIINFLAMATORIOS	8	\$29.99	4/4/2028
Producto Editado QA	QA	18	\$11.00	Sin fecha
Leche NanPro	SIN CATEGORIA	13	\$249.99	Sin fecha
Suerox	BEBIDAS	13	\$50.00	5/8/2028
Insulina				

Interfaz de inicio de sesión

The screenshot shows a login interface with a white card on a dark gray background. The card has a title 'INICIAR SESIÓN' and a close button (X). Below the title, there are two input fields: 'Usuario' and 'Contraseña'. At the bottom of the card, there's a link '¿No tienes cuenta? Regístrate' and a blue button labeled 'ENTRAR'.

Interfaz de compra por un invitado

The screenshot displays the VitalLogix Pharmacy Management System interface for a guest purchase. The browser address bar shows 'localhost' and the URL 'LuisGIDzib/VitalLogix: Pharmacy Management System. Java + PostgreSQL + React.' The interface is divided into two main sections: a product catalog on the left and a shopping cart summary on the right.

Product Catalog:

PRODUCTO	CATEGORIA	STOCK	PRECIO	VENCIMIENTO	ACCIÓN
Electrolit MED-0002	SUPLEMENTOS	8 UN.	\$15.00	4/9/2029	Añadir
Leche Nido CORREA	BEBIDAS	12 UN.	\$120.00	6/9/2031	Añadir
Ibuprofeno 1234	ANTIBIÓTICOS	4 UN.	\$120.00	6/5/2030	Añadir
paracetamol MED-0001	ANTIINFLAMATORIOS	0 UN.	\$29.99	4/4/2028	NO DISPONIBLE
Producto Editado QA QA-EDIT-001	QA	18 UN.	\$11.00	Sin fecha	Añadir
Leche NanPro MED-0003	SIN CATEGORIA	12 UN.	\$249.99	Sin fecha	Añadir
Suerox MED-0005	BEBIDAS	13 UN.	\$50.00	5/8/2028	Añadir
Insulina Inyectable INSU-001	CONTROLADOS	3 UN.	\$34.00	7/7/2027	Añadir

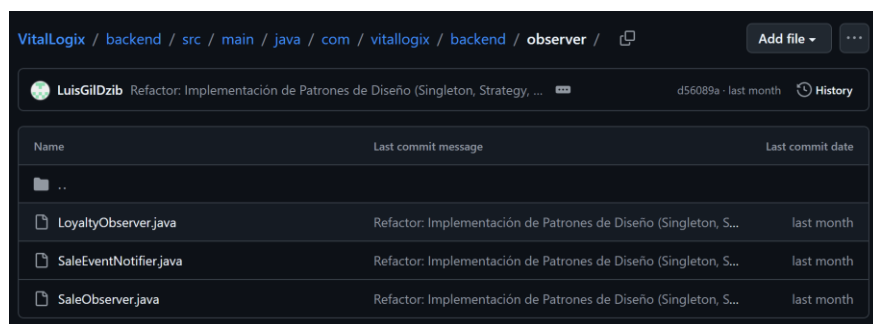
Shopping Cart Summary (RESUMEN DE VENTA):

- Ver recomendaciones personalizadas:** Basadas en los productos de tu carrito. [Ver sugerencias](#)
- VENTA CON RECETA:** ☐ **APLICAR CLIENTEAMIGO:** ☐
- SUEROX:** 1 unit, \$50.00. [Quitar](#)
- ELECTROLIT:** 1 unit, \$15.00. [Quitar](#)
- SUBTOTAL:** \$65.00
- TOTAL A PAGAR:** \$65.00
- [INICIA SESION PARA FINALIZAR COMPRA](#)
- Puedes explorar y agregar productos. El inicio de sesión se solicita al pagar.

Evidencias de implementación

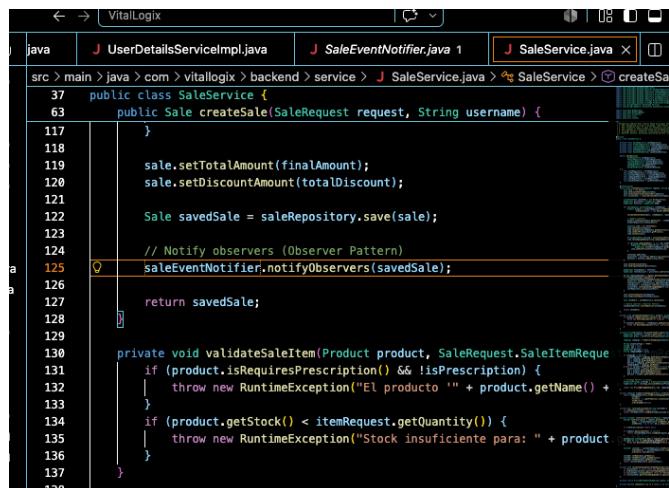
Patrón Observer

El objetivo es desacoplar la lógica de una venta de cualquier efecto secundario o acción que deba ocurrir después de que la venta se haya completado, LoyaltyObserver se encarga de revisar cuántas compras ha realizado un cliente y generar un código Cliente Amigo si llega a 5 compras. Debido a el patrón Observer, el SaleService no necesita saber nada sobre el sistema de lealtad (cupones), simplemente emite el evento: la venta terminó, y cualquier observador registrado reacciona de forma automática e independiente a este evento.



Name	Last commit message	Last commit date
...		
LoyaltyObserver.java	Refactor: Implementación de Patrones de Diseño (Singleton, S...	last month
SaleEventNotifier.java	Refactor: Implementación de Patrones de Diseño (Singleton, S...	last month
SaleObserver.java	Refactor: Implementación de Patrones de Diseño (Singleton, S...	last month

En la línea 125 de SaleService, una vez que esta finaliza sus tareas principales (cálculo de descuentos, cobro y guardado de la venta en la base de datos), invoca al Notificador (SaleEventNotifier). Este Notificador es el encargado de repartir el mensaje de que “la venta ha concluido” a todos los módulos interesados, activando así el LoyaltyObserver para que sume la compra al contador y asigne el cupón si se llega a 5 compras.

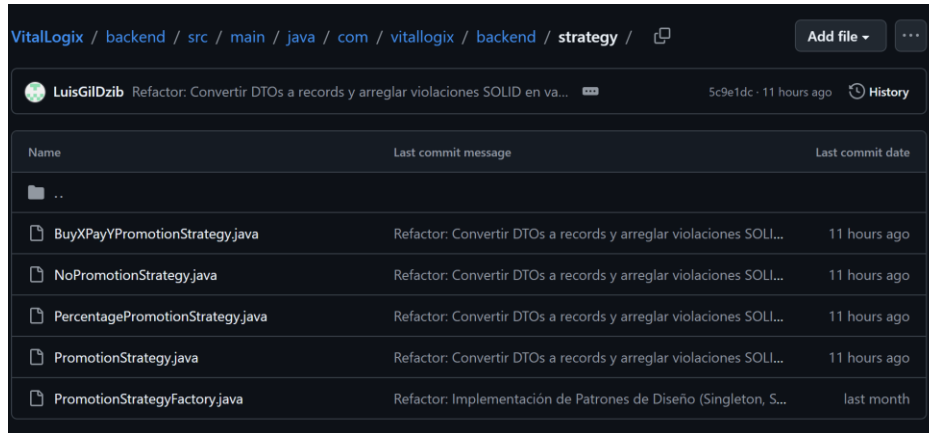


```

src > main > java > com > vitallogix > backend > service > J SaleService.java > SaleService > createSa
37 public class SaleService {
63 public Sale createSale(SaleRequest request, String username) {
117 }
118
119     sale.setTotalAmount(finalAmount);
120     sale.setDiscountAmount(totalDiscount);
121
122     Sale savedSale = saleRepository.save(sale);
123
124     // Notify observers (Observer Pattern)
125     saleEventNotifier.notifyObservers(savedSale);
126
127     return savedSale;
128
129
130 private void validateSaleItem(Product product, SaleRequest.SaleItemReque
131     if (product.isRequiresPrescription() && !isPrescription) {
132         throw new RuntimeException("El producto " + product.getName() +
133     }
134     if (product.getStock() < itemRequest.getQuantity()) {
135         throw new RuntimeException("Stock insuficiente para: " + product
136     }
137 }
138
  
```

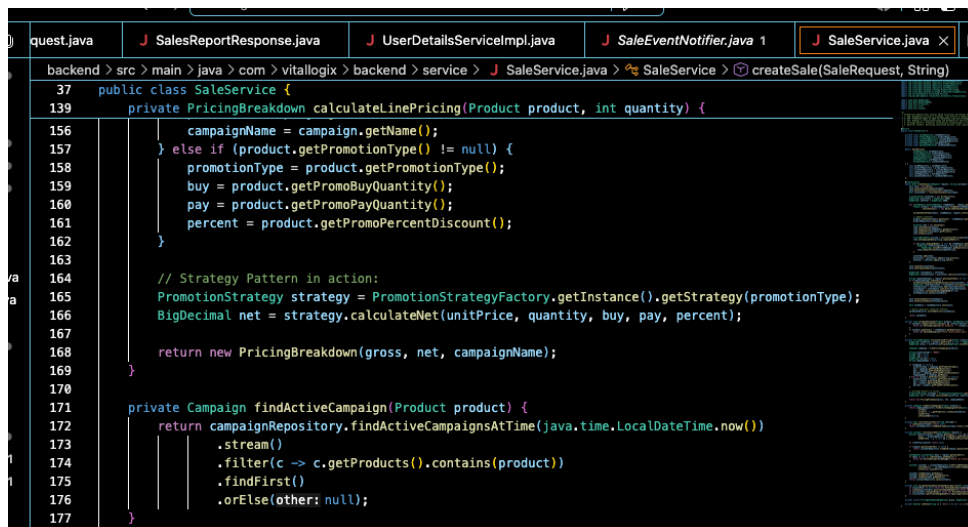
Patrón Strategy

El patrón Strategy nos permite cambiar dinámicamente el algoritmo de cálculo de precios. En lugar de llenar el proceso de ventas con un "if / else" para cada tipo de promoción, ponemos cada fórmulas en archivos separados. Así, si en el futuro queremos añadir una promoción nueva solo creamos un archivo nuevo y el sistema lo adopta, todo sin tener que modificar el código de SaleService, (SOLID Abierto/Cerrado).



Name	Last commit message	Last commit date
..		
BuyXPromotionStrategy.java	Refactor: Convertir DTOs a records y arreglar violaciones SOLI...	11 hours ago
NoPromotionStrategy.java	Refactor: Convertir DTOs a records y arreglar violaciones SOLI...	11 hours ago
PercentagePromotionStrategy.java	Refactor: Convertir DTOs a records y arreglar violaciones SOLI...	11 hours ago
PromotionStrategy.java	Refactor: Convertir DTOs a records y arreglar violaciones SOLI...	11 hours ago
PromotionStrategyFactory.java	Refactor: Implementación de Patrones de Diseño (Singleton, S...	last month

En estas líneas de SaleService (164-166), en lugar de hacer las matemáticas aquí mismo, el SaleService le pide a la Fábrica la estrategia de promoción correcta. Luego, simplemente le pasa los datos a esa estrategia y para que realice el cálculo y devuelva el precio neto, el servicio de ventas no conoce por completo cómo se hizo la matemática del descuento.



```

backend > src > main > java > com > vitallogix > backend > service > J SaleService.java > SaleService > createSale(SaleRequest, String)
37 public class SaleService {
139 private PricingBreakdown calculateLinePricing(Product product, int quantity) {
156     campaignName = campaign.getName();
157 } else if (product.getPromotionType() != null) {
158     promotionType = product.getPromotionType();
159     buy = product.getPromoBuyQuantity();
160     pay = product.getPromoPayQuantity();
161     percent = product.getPromoPercentDiscount();
162 }
163
164 // Strategy Pattern in action:
165 PromotionStrategy strategy = PromotionStrategyFactory.getInstance().getStrategy(promotionType);
166 BigDecimal net = strategy.calculateNet(unitPrice, quantity, buy, pay, percent);
167
168 return new PricingBreakdown(gross, net, campaignName);
169 }
170
171 private Campaign findActiveCampaign(Product product) {
172     return campaignRepository.findActiveCampaignsAtTime(java.time.LocalDateTime.now())
173         .stream()
174         .filter(c -> c.getProducts().contains(product))
175         .findFirst()
176         .orElse(null);
177 }
  
```

Patrón Singleton

En la línea 8 de PromotionStrategyFactory aplicamos el patrón Singleton, esto garantiza que solo se cree solo una fábrica en toda la memoria del servidor cuando arranca la aplicación. Como todas las ventas comparten esta única instancia, evitamos estar creando objetos innecesarios y ahorramos RAM.

```
backend > src > main > java > com > vitallogix > backend > strategy > PromotionStrategyFactory.java > PromotionStrateg
1 package com.vitallogix.backend.strategy;
2
3 import java.util.HashMap;
4 import java.util.Map;
5
6 public class PromotionStrategyFactory {
7     // Patrón Singleton
8     private static final PromotionStrategyFactory INSTANCE = new PromotionStrategyFactory();
9
10    private final Map<String, PromotionStrategy> strategies = new HashMap<>();
11
12    // Constructor privado para evitar instanciación externa
13    private PromotionStrategyFactory() {
14        strategies.put(key: "PERCENTAGE", new PercentagePromotionStrategy());
```

Ejemplo del funcionamiento del sistema

<https://youtu.be/D90ekEZowrc?si=Ifnjrp-dtbuOewSS>

Enlace al repositorio en GitHub

<https://github.com/LuisGilDzib/VitalLogix/tree/main>

Conclusiones

El proyecto de VitalLogix permitió realizar un sistema de gestión farmacéutica en producción capaz de dar respuesta de forma integral y completa a las operativas de una farmacia: control de stock, registro de venta de productos, gestión de clientes, gestión de categorías y generación de reportes.

A lo largo del proyecto se podía aplicar en forma concreta los principios SOLID y tres patrones de diseño. El patrón Observador desacopla el módulo de lealtad del flujo de venta de manera que el SaleService no necesite conocer el sistema de cupones para activarlo. El patrón Strategy favorecía la adherencia de nuevas clases de promoción sin necesidad de modificar el núcleo del Servicio de Venta cumpliendo de esta manera el principio Abierto/Cerrado. La implementación del patrón Singleton en PromotionStrategyFactory hizo posible que la fábrica desarrollada estuviese presente en memoria en una única instancia durante toda la vida de la aplicación, optimizando así el uso de la memoria.

La representación del sistema mediante diagramas UML demostró ser una gran aliada para poder comunicar y documentar las decisiones de diseño. El diagrama de clases expresa claramente el principio de separación de responsabilidades que existe entre controladores, servicios y repositorios. El diagrama de secuencia expresa de una forma ordenada el flujo de una venta, incluyendo las interacciones con el motor de sugerencias y el repositorio de clientes. El diagrama de estados del Producto permite ver los diferentes períodos de vida que puede tener una determinada referencia que esté en el inventario, teniendo presente stock, caducidad y visibilidad. El diagrama de colaboración y el de actividad permiten visualizar la vista dinámica del sistema y cómo se coordinan los objetos cuando se produce una venta.

A partir del proyecto realizado se puede inferir que aplicar desde el principio conceptos de diseño tales como SOLID no son una carga extra del desarrollo sino, al contrario, es una inversión que se traduce en facilitar el mantenimiento y la extensión del código. La separación entre interfaces e implementaciones concretas, algo que nos permite componentes como ReportServicePort, fue una base para poder modificar y probar módulos de forma individual, sin afectar al sistema.